

Zewail City, University of Science and Technology
Program of Communications and Information Engineering
CIE-349 / CIE-408 — Embedded Systems, Spring 2026
Instructors: Hazem Fahmy, PhD George Maximous, MSc
TAs: Zaynap Ahmed, BSc Yasser Deif, BSc

Advanced Driver Assistance System (ADAS) — Embedded Cyber-Physical Vehicle Final Project Report

Spring 2026

Team Member:

Mohammed Ali Sadek

Submitted in partial fulfillment of the requirements for CIE-349 / CIE-408

Repository:

<https://github.com/MohammedAliSadek/ADAS-Embedded-System-Project>

Firebase DB:

<https://adas-driver-dashboard-default-rtdb.firebaseio.com/>

HuggingFace Model & Dataset:

<https://huggingface.co/MohammedAliSadek/adas-lane-detection>

Contents

1	Introduction and Literature Review	2
1.1	Project Overview	2
1.2	Literature Review	2
2	Components and Input/Output Diagram	2
2.1	Bill of Materials	2
2.2	Input/Output Diagram	3
3	System Architecture	3
3.1	Three-Node Hardware Architecture	3
3.2	PIC16F877A Pin Assignment	4
3.3	Pin Assignment Table	5
3.4	Firmware Layered Architecture	5
3.5	Interrupt Architecture	5
4	Safety-Critical Features (PIC16F877A)	6
4.1	F1 — Forward Collision Warning and AEB	6
4.2	F2 — Automatic Door Safety Interlock	6
4.3	F3 — Emergency Stop with Toggle Interlock	7
4.4	F4 — Automatic Headlight Control (LDR with Hysteresis)	7
5	Raspberry Pi Software: Machine Learning Pipeline	8
5.1	Phase 1 — Hardware Verification (<code>cam_test.py</code>)	8
5.2	Phase 2 — In-Situ Dataset Engineering (<code>collect_data.py</code>)	8
5.3	Phase 3 — Edge Inference and Flask Micro-service (<code>live.py</code>)	9
5.4	Phase 4 — Cloud Telemetry Bridge (<code>bridge.py</code>)	9
5.5	Phase 5 — Model Training and HuggingFace Publication	10
6	System Automation: systemd Daemonisation	10
6.1	Vision Service (<code>lane.service</code>)	11
6.2	Cloud Bridge Service (<code>firebase-bridge.service</code>)	11
6.3	Boot Dependency Sequencing	11
7	Mobile Application and Firebase Integration	12
7.1	Firebase Database Structure	12
7.2	Flutter Application	12
7.3	Firebase Security Rules	12
8	Key Bug Fixes and Design Decisions	13
9	Replicability	13
9.1	Microcontroller Firmware	13
9.2	Raspberry Pi	14
9.3	Mobile Application	14
10	Results and Demonstration Summary	15
A	Raspberry Pi File Inventory	15
B	Glossary	15

1 Introduction and Literature Review

1.1 Project Overview

This project implements a complete Advanced Driver Assistance System (ADAS) on a four-wheel-drive autonomous car platform built around a **PIC16F877A** microcontroller at 20 MHz, co-processed by a **Raspberry Pi 4B** running a TensorFlow Lite lane-detection pipeline. The system integrates real-time sensing, interrupt-driven safety logic, cloud telemetry via Google Firebase, and a Flutter mobile dashboard into a single cyber-physical vehicle demonstrator.

The embedded firmware follows a strict three-layer architecture: **MCAL** $\rightarrow$ **HAL** $\rightarrow$ **Application**. The Raspberry Pi runs two decoupled micro-services (**live.py** and **bridge.py**) that communicate through a local Flask HTTP API, ensuring the vision pipeline is never stalled by network I/O.

1.2 Literature Review

Forward Collision Warning / AEB. ISO 22737 and Euro NCAP require FCW at $TTC \leq 2$ s and AEB at $TTC \leq 1$ s. Work on low-cost MCU implementations [2] shows ultrasonic sensors are sufficient for low-speed scenarios (<30 km/h) at ≥ 6 Hz measurement rate.

Lane Keeping Assist. Starting from LaneNet [3], CNN-based lane detectors have been compressed to TFLite classifiers (e.g. Ultra-Fast Lane Detection [4], GitHub) runnable on embedded Linux at real-time frame rates. In-situ data collection is preferred over public datasets to eliminate domain shift [5].

Embedded Safety Architectures. IEC 61508 and ISO 26262 mandate hardware/software partitioning. On PIC MCUs, interrupt-driven architectures with GIE-off critical sections substitute for multi-core lockstep [5].

IoT Vehicle Telemetry. Firebase Realtime Database [6] provides <200 ms cloud round-trips suitable for vehicle telemetry at ≤ 10 Hz, well within our 6.7 Hz main-loop cadence and 10 Hz bridge polling rate.

2 Components and Input/Output Diagram

2.1 Bill of Materials

Table 1: Hardware components

Component	Role	Interface
PIC16F877A (40-DIP)	Central MCU, 20 MHz HS crystal	—
Raspberry Pi 4B	TFLite lane inference, Firebase bridge	UART / CSI
HC-SR04 Ultrasonic	Front obstacle distance — FCW/AEB (F1)	RA1 (TRIG), RA2 (ECHO)
LDR + resistor	Ambient light — auto headlights (F4)	AN0 (ADC CH0)
L298N Dual H-Bridge	4WD motor driver	RD0–RD3, RC1, RC2
Active Buzzer	Audible warning patterns	RC5
PCF8574 I2C LCD	16×2 HD44780 status display	I2C RC3/RC4
Magnetic Reed Switch	Door-open detection — RBIF (F2)	RB4
Tactile Push-Button	Emergency-stop toggle — EXT INT (F3)	RB0
Red LED	Door-1-open indicator	RB1
Green LED	Heartbeat / all-OK indicator	RB3
White LEDs (×4)	Automatic headlights	RD4–RD7
Pi Camera V2	Road-scene images for TFLite model	CSI ribbon

Note: *Reserved / not wired.* RB2 (Door-2 LED), RB5 (Door-2 sensor), and RC0 (servo output) are defined in firmware but physically disconnected in the current prototype (marked *n/c* in Figure 2). Component photographs for all hardware items above are provided in the project [GitHub repository README](#).

2.2 Input/Output Diagram

Figure 1 shows the top-level data-flow diagram for the ADAS system.

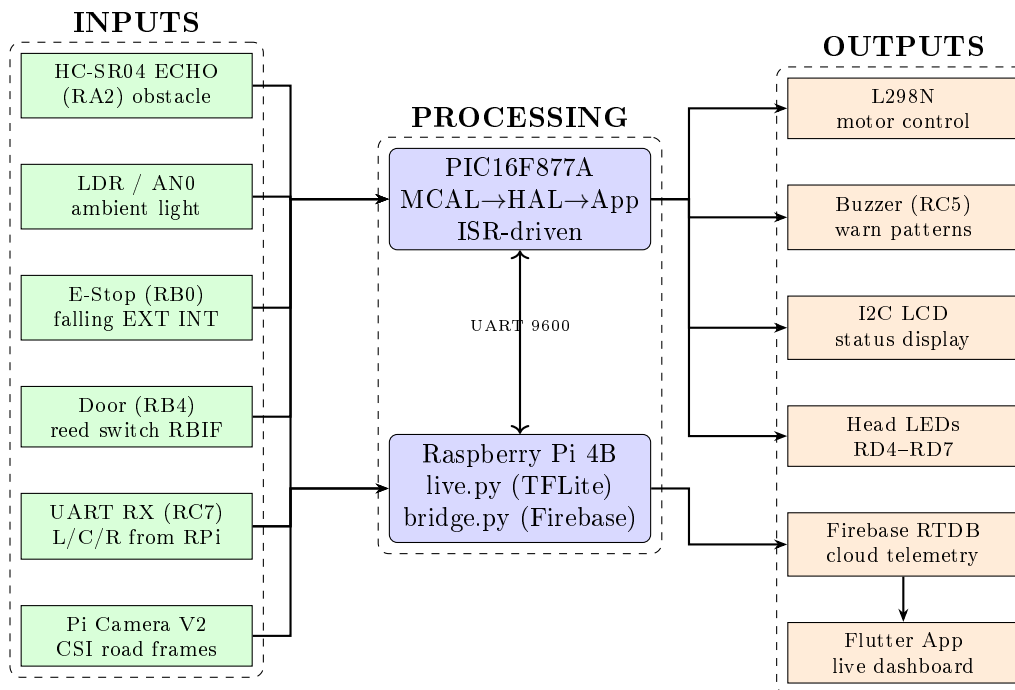


Figure 1: Top-level ADAS system input/output diagram.

3 System Architecture

3.1 Three-Node Hardware Architecture

1. **PIC16F877A** — real-time control node. Runs all MCAL/HAL firmware, handles four interrupt sources, drives motors and actuators, streams telemetry at 9600 baud.
2. **Raspberry Pi 4B** — perception and connectivity node. Runs two decoupled Python services (`live.py` and `bridge.py`), manages the Pi Camera V2, publishes data to Firebase over Wi-Fi, and forwards lane commands to the PIC via UART.
3. **Firebase Realtime Database** — cloud bus. Two write paths: `bridge.py` writes `laneState`, `laneConfidence`, and `lastUpdate` directly to the database root (/); the RPi UART parser writes the full vehicle telemetry to `vehicles/car_01`, which the Flutter app subscribes to.

3.2 PIC16F877A Pin Assignment

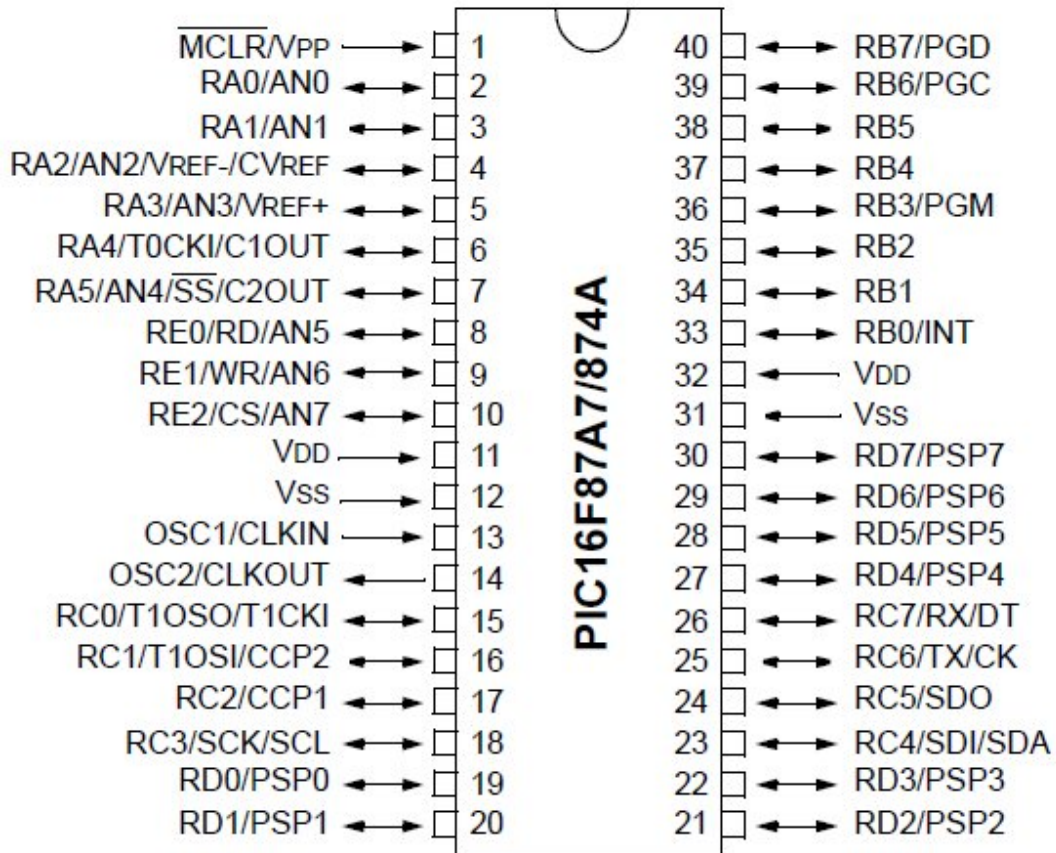


Figure 2: PIC16F877A ADAS pin assignment (40-DIP, 20 MHz HS). Colour key: **red** = power/GND, **teal** = ADC/Analog, **green** = GPIO/Digital, **orange** = Timer/PWM, **purple** = Communication, **yellow** = Interrupt/Switch. Pins RB2, RB5, RC0 are reserved in firmware but not physically wired (*n/c*). Pull-ups on PORTB enabled via `OPTION_REG`. Final `ADCON1 = 0x8E`: AN0 analog, AN1–AN7 digital, right-justified.

3.3 Pin Assignment Table

Table 2: Complete PIC16F877A pin assignment (consistent with Figure 2)

Pin	Label	Dir.	Config	Function
2	RA0/AN0	IN	Analog	LDR — ADC Channel 0
3	RA1	OUT	Digital	HC-SR04 TRIG
4	RA2	IN	Digital	HC-SR04 ECHO (ADCON1 = 0x8E)
5–7	RA3–5	IN	—	Unused (inputs)
19	RD0	OUT	Digital	L298N IN1 (Motor A)
20	RD1	OUT	Digital	L298N IN2 (Motor A)
21	RD2	OUT	Digital	L298N IN3 (Motor B)
22	RD3	OUT	Digital	L298N IN4 (Motor B)
23	RC4/SDA	I2C	Open-dr.	LCD I2C data
24	RC5	OUT	Digital	Buzzer (active HIGH)
25	RC6/TX	OUT	UART	UART TX to Raspberry Pi
26	RC7/RX	IN	UART	UART RX from Raspberry Pi
27–30	RD4–7	OUT	Digital	Headlight LEDs 1–4
33	RB0/INT	IN	INT	Emergency-stop button (falling-edge)
34	RB1	OUT	Digital	Red LED — Door 1 indicator
35	RB2	—	<i>n/c</i>	Reserved: Door 2 LED
36	RB3	OUT	Digital	Heartbeat / OK LED
37	RB4/RBIF	IN	RBIF	Door reed switch
38	RB5	—	<i>n/c</i>	Reserved: Door 2 sensor
39	RB6/PGC	IN	Digital	ICSP clock
40	RB7/PGD	IN	Digital	ICSP data
15	RC0	—	<i>n/c</i>	Reserved: servo soft-PWM
16	RC1/CCP2	OUT	PWM	Motor ENB (PWM Ch2)
17	RC2/CCP1	OUT	PWM	Motor ENA (PWM Ch1)
18	RC3/SCL	I2C	Open-dr.	LCD I2C clock
11,32	VDD	+5V	PWR	Supply voltage
12,31	VSS	GND	PWR	Ground
13,14	OSC1/2	OSC	—	20 MHz HS crystal
1	MCLR/VPP	IN	—	Reset / ICSP voltage

3.4 Firmware Layered Architecture

Table 3: Firmware source-code layers

Layer	Folder	Responsibility
Application	APP/main.c	System init, main loop, ISR callbacks
HAL	HAL/{MOTOR, BUZZER, ...}/	Sensor/actuator abstractions
MCAL	MCAL/{GPIO, ADC, PWM, ...}/	Register-level peripheral drivers
Services	SERVICES/	STD_TYPES, BIT_MATH, MCU_CONFIG

3.5 Interrupt Architecture

The single interrupt vector (`__interrupt() isr()`) in `Interrupt_Manager.c` multiplexes four sources:

1. **Timer0 overflow** ($\approx 204.8 \mu\text{s}$, prescaler 1:4) — Drives `BUZZER_Tick()`.
2. **EXT INT / RB0 falling edge** — toggles `g_estop_active`, calls `MOTOR_Stop()` immediately.

3. **UART RX** ($\text{RCIF} \wedge \text{RCIE}$) — stores lane byte in `g_lane_cmd`.
4. **RBIF** $\wedge$ **RBIE — door-edge on RB4; calls `DOOR_SENSOR_ISR()`.**

4 Safety-Critical Features (PIC16F877A)

4.1 F1 — Forward Collision Warning and AEB

Hardware. HC-SR04 on RA1 (TRIG) / RA2 (ECHO). Timer1 prescaler 1:8 gives $1.6\mu\text{s}/\text{tick}$, 36 ticks/cm. `ADCON1 = 0x07` in `ULTRASONIC_Init()` makes RA2 digital; `ADC_Init()` subsequently restores `ADCON1 = 0x8E`.

Zone	Distance	Speed	Warning	Control
CLEAR	>50 cm	70%	Buzzer OFF	Forward 70%
WARN	30–50 cm	40%	Slow beep	Forward 40%
SLOW	10–30 cm	15%	Fast beep	Forward 15%
STOP	≤10 cm	0%	Continuous buzz	<code>MOTOR_Stop()</code>

```

1  dist = ULTRASONIC_GetDistance(); // cm | 400 (out-of-range) | 401 (no sensor
   )
2
3  if dist >= OUT_OF_RANGE: zone = CLEAR    // fault -> treat as clear
4  elif dist > 50:          zone = CLEAR
5  elif dist > 30:          zone = WARN
6  elif dist > 10:          zone = SLOW
7  else:                    zone = STOP
8
9  switch zone:
10   CLEAR: MOTOR_Forward(70%); BUZZER_Off();      LCD "[CLEAR] Spd:70%"
11   WARN:  MOTOR_Forward(40%); BUZZER_SlowBeep(); LCD "[WARN ] Spd:40%"
12   SLOW:  MOTOR_Forward(15%); BUZZER_FastBeep(); LCD "[SLOW ] Spd:15%"
13   STOP:  MOTOR_Stop();      BUZZER_Continuous(); LCD "[STOP ] Spd:00%"

```

Listing 1: FCW/AEB zone decision pseudocode

Sentinels 400 and 401 both exceed all FCW thresholds, so a faulty or disconnected sensor never silently triggers false braking. Both are displayed on the LCD as `D:--` or `NOSENSOR`.

4.2 F2 — Automatic Door Safety Interlock

Hardware. Reed switch on RB4 (RBIF). PORTB pull-ups enabled (`RBPUE=0`). RB1 = red indicator LED.

```

1  // ISR (RBIF fires on RB4 edge):
2  DOOR_SENSOR_ISR():
3      door_now = GPIO_GetPinValue(RB4) // reading PORTB clears RBIF mismatch
4      if door_now != door_prev:
5          update RB1 indicator LED
6          Door_Callback(1, door_now == DOOR_OPEN)
7
8  // Callback (inside ISR):
9  Door_Callback(doorNum, isOpen):
10     g_door_open = isOpen; g_door_event = 1
11     if isOpen: MOTOR_Stop(); pretrip_ok = 0 // CONTROL: immediate stop
12
13 // Main loop:
14 if g_door_event:
15     if g_door_open: BUZZER_SlowBeep(); LCD "DOOR OPEN! WAIT" // WARNING

```

```

16     else:                pretrip_ok=1; BUZZER_Off(); LCD "DOOR OK GO!"
17
18     // Safety gate every iteration:
19     if NOT pretrip_ok OR g_estop_active: MOTOR_Stop(); continue

```

Listing 2: Door interlock pseudocode

4.3 F3 — Emergency Stop with Toggle Interlock

Hardware. Tactile button on RB0, falling-edge EXT INT.

```

1 EmergencyStop_Callback():           // runs in EXT_INT ISR
2     g_estop_active ^= 1             // toggle on each press
3     g_estop_event = 1
4     if g_estop_active: MOTOR_Stop() // CONTROL: immediate stop
5
6     // Main loop event handler:
7     if g_estop_event:
8         if g_estop_active:
9             BUZZER_SingleBeep()      // WARNING: one 200 ms beep
10            LCD "** E-STOP **"; UART "ESTOP\n"
11        else:
12            BUZZER_Off(); LCD "E-STOP CLEARED"; UART "RESM\n"

```

Listing 3: Emergency stop toggle pseudocode

4.4 F4 — Automatic Headlight Control (LDR with Hysteresis)

Hardware. LDR on AN0/RA0. Final ADCON1 = 0x8E: 10-bit, right-justified. Four white LEDs on RD4–RD7.

Two-Level Output Design. F4 implements a three-zone state machine that satisfies the mandatory two-output-level requirement: a *warning* output (dimmed headlights + single audible beep) precedes the final *control* output (full headlights ON), with symmetric hysteresis on every transition to prevent flicker.

```

1 // Thresholds (0-1023, LOW = bright, HIGH = dark):
2 LDR_DIM_ENTER = 462 // enter WARNING zone: dim headlights + beep
3 LDR_FULL_ENTER = 562 // enter CONTROL zone: full headlights ON
4 LDR_DIM_EXIT = 362 // exit WARNING back to LIT (hysteresis dead-band)
5 LDR_FULL_EXIT = 462 // exit CONTROL back to WARNING (= DIM_ENTER)
6
7 // State: LIT -> WARNING -> DARK (and reverse with hysteresis)
8
9 ldr_val = LDR_Read() // 0..1023
10
11 // --- Transitions into darker states ---
12 if (state == LIT) AND (ldr_val > LDR_DIM_ENTER):
13     state = WARNING
14     LDR_HeadlightsDim() // OUTPUT LEVEL 1 WARNING:
15     BUZZER_SingleBeep() // dim LEDs (50% duty) + one beep
16     LCD "L:DIM "; UART "HL:DIM"
17
18 if (state == WARNING) AND (ldr_val > LDR_FULL_ENTER):
19     state = DARK
20     LDR_HeadlightsFull() // OUTPUT LEVEL 2 CONTROL:
21     BUZZER_Off() // full LEDs ON (100% duty)
22     LCD "L:DARK"; UART "HL:ON"
23
24 // --- Transitions into lighter states ---
25 if (state == DARK) AND (ldr_val < LDR_FULL_EXIT):
26     state = WARNING

```

```

27     LDR_HeadlightsDim()           // Back to WARNING level
28     BUZZER_SingleBeep()
29     LCD "L:DIM "; UART "HL:DIM"
30
31     if (state == WARNING) AND (ldr_val < LDR_DIM_EXIT):
32         state = LIT
33         LDR_HeadlightsOff()        // Headlights fully OFF
34         LCD "L:LIT "; UART "HL:OFF"

```

Listing 4: Three-zone headlight control with warning level

5 Raspberry Pi Software: Machine Learning Pipeline

The Raspberry Pi 4B hosts a complete five-phase ML and telemetry pipeline, implemented as a set of standalone Python scripts managed by systemd. The inference engine is TensorFlow Lite; the model classifies each camera frame into three lane states: `safe`, `warning`, `off_track`.

5.1 Phase 1 — Hardware Verification (`cam_test.py`)

Before deploying the ML pipeline, the camera module is validated in isolation. The test script:

1. Initialises the `libcamera`-based `Picamera2` driver at 640×480 .
2. Applies a hardware-level ISP transform `Transform(hflip=True, vflip=True)` — zero CPU overhead.
3. Waits 2s for AEC/AWB convergence.
4. Writes `test_shot.jpg` as a human-inspectable artefact.

```

1  picam2 = Picamera2()
2  config = picam2.create_still_configuration(
3      main={"size": (640, 480)},
4      transform=Transform(hflip=True, vflip=True) # ISP-level, zero CPU cost
5  )
6  picam2.configure(config)
7  picam2.start()
8  time.sleep(2) # AEC/AWB convergence
9  picam2.capture_file("test_shot.jpg")
10 picam2.stop()

```

Listing 5: `cam_test.py` — key initialisation

5.2 Phase 2 — In-Situ Dataset Engineering (`collect_data.py`)

Training data is collected in situ using the exact same camera, mounting orientation, field of view, and road environment that the deployed model will encounter. Off-the-shelf lane datasets introduce domain shift that degrades real-world accuracy.

```

1  base_path = "lane_dataset"
2  categories = ["safe", "warning", "off_track"]
3  for cat in categories:
4      os.makedirs(os.path.join(base_path, cat), exist_ok=True)

```

Listing 6: Dataset directory scaffold

Table 4: Three-class labelling taxonomy

Key	Class	Semantic Definition	ADAS Action
s	safe	Vehicle fully within lane markings	No alert
w	warning	Wheel touching or crossing lane boundary	Audio/haptic alert
o	off_track	Vehicle fully outside lane markings	Emergency alert

Each captured frame is named using a Unix epoch float timestamp (`time.time()`): `lane_dataset/<class>/<timestamp>`. This guarantees lexicographic ordering, collision-free rapid capture, and an embedded temporal record for later dataset analysis.

5.3 Phase 3 — Edge Inference and Flask Micro-service (`live.py`)

```

1 raw = picam2.capture_array()      # DMA -> numpy array; zero disk I/O
2 img = Image.fromarray(raw)        # zero-copy: no intermediate buffer
3 img = img.resize((128, 128))      # bilinear resize to model input

```

Listing 7: RAM-capture pipeline (no SD writes)

Table 5: TFLite inference pipeline steps in `live.py`

Step	Operation	Detail
1	RAM Buffer Acquisition	<code>capture_array()</code> — DMA, zero disk I/O
2	Spatial Normalisation	<code>Image.resize((128,128))</code> bilinear
3	Pixel Scaling	<code>np.array(..., float32) / 255.0</code> : $[0, 255] \rightarrow [0.0, 1.0]$
4	Batch Dimension	<code>np.expand_dims(..., 0)</code> : $(128, 128, 3) \rightarrow (1, 128, 128, 3)$
5	TFLite Inference	<code>interpreter.invoke()</code> — quantised model
6	argmax Decoding	<code>CLASSES[np.argmax(probs)]</code> : index $\rightarrow$ label

```

1 camera_thread = threading.Thread(target=vision_loop, daemon=True)
2 camera_thread.start()
3 app.run(host='0.0.0.0', port=5000, threaded=True)
4
5 # /state endpoint (stateless, 0(1), no computation):
6 @app.route('/state', methods=['GET'])
7 def state():
8     return jsonify(current_state) # {"state":..., "confidence":..., "fps":...}

```

Listing 8: Daemon-thread producer + Flask consumer

5.4 Phase 4 — Cloud Telemetry Bridge (`bridge.py`)

```

1 last_sent_state = None # cache of most recently pushed state
2
3 while True:
4     resp = requests.get("http://localhost:5000/state", timeout=1)
5     state = resp.json().get('state')
6
7     if state != last_sent_state: # only write on genuine transition
8         db_reference.update({
9             "laneState": state,
10            "laneConfidence": round(confidence, 3),

```

```

11         "lastUpdate":      int(time.time() * 1000) # Unix ms timestamp
12     })
13     last_sent_state = state          # update cache
14
15     time.sleep(0.1)                 # 10 polls / second

```

Listing 9: `bridge.py` delta-tracking loop

The delta-tracking pattern ensures that a sustained **safe** state produces **zero** Firebase writes. Only a genuine lane-state transition triggers exactly one write, reducing Firebase operations by orders of magnitude versus a naive always-write implementation.

```

1 cred = credentials.Certificate(FIREBASE_KEY_PATH)
2 firebase_admin.initialize_app(cred, {'databaseURL': FIREBASE_DB_URL})
3 db_reference = db.reference('/') # partial update; does not erase other
    keys

```

Listing 10: Firebase Admin SDK initialisation

5.5 Phase 5 — Model Training and HuggingFace Publication

Model Architecture. The TFLite classifier uses a lightweight MobileNetV2 backbone (ImageNet pre-trained) with a custom 3-class softmax head, fine-tuned on the in-situ dataset collected by `collect_data.py`. Input resolution is $128 \times 128 \times 3$ (`float32`, normalised to $[0, 1]$). The full training pipeline — including data augmentation, class-weight balancing, accuracy/loss curves, confusion matrix, and per-class precision/recall — is documented in the `lane_training.ipynb` notebook in the project repository.

Table 6: Model configuration summary (`model_config.json`)

Parameter	Value
Backbone	MobileNetV2 (ImageNet pre-trained, top removed)
Head	GlobalAveragePooling2D → Dense(128, ReLU) → Dense(3, Softmax)
Input shape	$128 \times 128 \times 3$, <code>float32</code> , $[0.0, 1.0]$
Classes	<code>safe</code> , <code>warning</code> , <code>off_track</code>
Optimizer	Adam, $lr = 1 \times 10^{-4}$
Epochs	30 (early stopping, patience = 5)
Batch size	32
Data split	80 % train / 10 % val / 10 % test
Deployment format	TFLite (<code>lane_model.tflite</code>), <code>float32</code>
Inference FPS on Pi	≈ 15 FPS

HuggingFace. The trained model weights (`lane_model.tflite`), the in-situ dataset, and the training notebook are published at:

<https://huggingface.co/MohammedAliSadek/adas-lane-detection>

This URL is also cited in the project `README.md` as required by the project specification (Requirement 4).

6 System Automation: systemd Daemonisation

Both `live.py` and `bridge.py` are managed as `systemd` services, enabling automatic startup on vehicle power-on, dependency-ordered boot sequencing, and self-healing restart behaviour.

6.1 Vision Service (lane.service)

```

1 [Unit]
2 Description=ADAS Lane Detection Service
3 After=network.target
4
5 [Service]
6 ExecStart=/usr/bin/python3 /home/pi/adas/live.py
7 WorkingDirectory=/home/pi/adas
8 Restart=always
9 RestartSec=5
10 User=pi
11
12 [Install]
13 WantedBy=multi-user.target

```

Listing 11: /etc/systemd/system/lane.service

6.2 Cloud Bridge Service (firebase-bridge.service)

```

1 [Unit]
2 Description=Firebase Telemetry Bridge Service
3 After=network.target lane.service
4 Requires=lane.service
5
6 [Service]
7 ExecStart=/usr/bin/python3 /home/pi/adas/bridge.py
8 WorkingDirectory=/home/pi/adas
9 Restart=always
10 RestartSec=5
11 User=pi
12
13 [Install]
14 WantedBy=multi-user.target

```

Listing 12: /etc/systemd/system/firebase-bridge.service

6.3 Boot Dependency Sequencing

`After=network.target` ensures both services wait for the network subsystem. The `Requires=lane.service` directive in the bridge unit guarantees the Flask API is accepting connections before `bridge.py` begins polling, preventing an error-retry loop on boot. `Restart=always` with `RestartSec=5` provides self-healing within 5s without operator intervention.

```

1 sudo systemctl enable lane.service firebase-bridge.service # boot enable
2 sudo systemctl start lane.service firebase-bridge.service # start now
3 sudo systemctl status lane.service # check state
4 sudo journalctl -u lane.service -f # live logs
5 sudo journalctl -u firebase-bridge.service -f

```

Listing 13: Service management quick reference

7 Mobile Application and Firebase Integration

7.1 Firebase Database Structure

Table 7: Firebase write paths

Path	Writer	Fields
/ (root)	bridge.py (RPi)	laneState, laneConfidence, lastUpdate
/vehicles/car_01	RPi UART parser	All telemetry fields (read by Flutter app)
/vehicles/car_01	Flutter app	position, commandStatus, commandUpdatedAt

Both writers use `.update()` (partial writes) so neither path overwrites the other.

Table 8: `vehicles/car_01` fields (read by Flutter dashboard)

Field	Type	Writer	Description
online	bool	RPi heartbeat	Controller reachable
distanceCm	int	RPi UART parser	HC-SR04 reading (cm)
motorSpeed	int	RPi UART parser	PWM duty % (0–80)
fcwState	string	RPi UART parser	CLEAR/WARN/SLOW/STOP
laneState	string	bridge.py	safe/warning/off_track
laneConfidence	float	bridge.py	Softmax prob (0–1)
lightOn	bool	RPi UART parser	Headlights active
pretripOk	bool	RPi UART parser	Pre-trip gate passed
emergencyStop	bool	RPi UART parser	E-stop latch
door1Open	bool	RPi UART parser	Reed switch state
position	string	Flutter app	left/center/right
commandStatus	string	Flutter / RPi	pending → done
commandUpdatedAt	ts	Flutter app	<code>ServerValue.timestamp</code>
lastUpdate	ts	bridge.py	Unix ms timestamp

7.2 Flutter Application

The Flutter app uses `StreamBuilder<DatabaseEvent>` on `vehicleRef.onValue` to live-rebuild the dashboard.

```

1 Future<void> sendServoCommand(String position) async {
2   await vehicleRef.update({
3     'position':      position,    // 'left', 'center', or 'right'
4     'commandStatus': 'pending',
5     'commandUpdatedAt': ServerValue.timestamp,
6   });
7 }
8 // RPi reads commandStatus == 'pending', executes, writes 'done'

```

Listing 14: Flutter `sendServoCommand`

7.3 Firebase Security Rules

```

1 {
2   "rules": {
3     ".read": true,
4     ".write": false,
5     "vehicles": {
6       "car_01": { ".read": true, ".write": true }

```

```

7     }
8   }
9 }

```

Listing 15: Firebase Realtime Database security rules

Root-level writes from unknown clients are blocked; only `vehicles/car_01` is writable from the mobile app. `bridge.py` authenticates via service-account credentials that are granted write access to the root separately through IAM.

Security Note: The current rule `".read": true` at the database root makes all telemetry fields globally readable without authentication. For production deployment this should be tightened to require authentication (`".read": "auth != null"`). The open read-access is acceptable for a course demonstrator but should be noted as a known limitation.

8 Key Bug Fixes and Design Decisions

Table 9: Notable bugs discovered and fixed during development

Module	Bug and Resolution
TIMER0	TOCS_BIT defined as bit 4; corrected to bit 5 per datasheet Table 2-1. Wrong value caused Timer0 to use RA4 as external clock source.
TIMER1	TMR1L/H/T1CON raw-pointer macros clashed with <code>xc.h</code> SFR declarations, silently corrupting Timer1 in <code>ULTRASONIC.c</code> (null distance readings). Fixed by removing macros; use <code>xc.h</code> directly.
ULTRASONIC	(1) Old timeout <code>delay_us(10) × 500 = 5ms</code> too short for slow sensor clones. Replaced with two-pass Timer1 overflow ($\approx 104\text{ms}$). (2) Old fault return was 0cm, identical to clear road; replaced with sentinels 400/401 which exceed all FCW thresholds.
BUZZER	Race condition: setter wrote three volatile variables in separate stores; ISR could read an inconsistent mix. Fixed with <code>BUZZER_ENTER_CRITICAL()</code> GIE-off block wrapping all three writes atomically.
LCD I2C	(1) 100kHz corrupted nibble transfers on cheap PCF8574 boards. Lowered to 50kHz. (2) Missing <code>LCD_CMD_DISPLAY_OFF</code> before <code>LCD_CMD_CLEAR</code> violates HD44780 power-on sequence. (3) <code>LCD_SendNibble</code> used implicit nibble shifting; replaced with explicit D4–D7 bit names.
ISR UART	RCIF is a hardware status flag that sets regardless of RCIE. Without the enable-bit guard, phantom <code>UART_ISR()</code> calls fired when RX interrupts were disabled. Fixed: check both RCIF and RCIE.
PORTC RMW	Timer0 ISR read-modify-write on PORTC while MSSP held SDA/SCL LOW caused bus stall. Fixed: GIE-off block spans entire Start→Write→Stop with explicit RC3/RC4 latch-clear. Window $\approx 600\mu\text{s}$ (3 Timer0 ticks).
MOTOR	100% PWM overheated 5V supply wires. <code>MOTOR_SPEED_MAX</code> capped at 80% until wiring gauge is upgraded.
ADCON1	<code>ULTRASONIC_Init()</code> writes <code>ADCON1 = 0x07</code> , clearing the ADFM bit set by <code>ADC_Init()</code> . Fix: call <code>ULTRASONIC_Init()</code> before <code>LDR_Init()</code> so ADC init runs last, leaving <code>ADCON1 = 0x8E</code> .

9 Replicability

9.1 Microcontroller Firmware

1. Install MPLAB X IDE v6.x and XC8 v2.x.

2. Open `PIC_DRIVERS/` targeting PIC16F877A at 20 MHz HS.
3. In `APP/main.c`: `RUN_APPLICATION = 1` for full ADAS mode, or `RUN_LCD_ULTRASONIC_TEST = 1` for hardware diagnostics.
4. Build and program via PICKit 3/4 (LVP = OFF, HV on MCLR).

9.2 Raspberry Pi

1. Flash Raspberry Pi OS (64-bit Lite).
2. `git clone https://github.com/MohammedAliSadek/ADAS-Embedded-System-Project`
3. `pip install -r requirements.txt --break-system-packages`
4. Place `firebase_credentials.json` (GCP service-account key) in `/home/pi/adas/`.
5. Download the pre-trained TFLite model from HuggingFace:
<https://huggingface.co/MohammedAliSadek/adas-lane-detection>
 and place `lane_model.tflite` in `/home/pi/adas/`.
6. Run `python3 cam_test.py` — validate camera and save `test_shot.jpg`.
7. (*Optional*) Run `python3 collect_data.py` to gather in-situ training frames (keys: `s/w/o`).
8. (*Optional*) To retrain the model, open and run the `lane_training.ipynb` notebook. The notebook documents the full fine-tuning pipeline, data splits, training curves, confusion matrix, and per-class accuracy metrics.
9. Copy `lane.service` and `firebase-bridge.service` to `/etc/systemd/system/`; then:


```
sudo systemctl enable lane.service firebase-bridge.service
sudo systemctl start lane.service firebase-bridge.service
```
10. Docker image: `docker pull <registry>/adas-rpi:latest`

9.3 Mobile Application

1. Install Flutter SDK 3.x.
2. `cd mobile_app && flutter pub get`
3. Copy `google-services.json` from the Firebase console.
4. `flutter run (debug)` or `flutter build apk (release)`.

10 Results and Demonstration Summary

Table 10: Feature verification checklist

Feature	Behaviour verified	Status
F1 FCW/AEB	4-zone response; <code>MOTOR_Stop</code> at ≤ 10 cm; buzz patterns match zone	
F2 Door Interlock	Immediate stop on open; re-engage on confirmed close	
F3 Emergency Stop	Toggle engage/release each press; single 200 ms beep on engage	
F4 Auto Headlights	3-state (LIT/DIM/FULL); warning beep at DIM; zero flicker	
LCD display	16-char rows update on ≥ 2 cm or zone/light change	
UART telemetry	D:NNN L:LIT HL:OFF S:70 N:C\n at 9600 baud	
<code>cam_test.py</code>	Camera init, hflip/vflip, AEC warm-up, <code>test_shot.jpg</code> saved	
<code>collect_data.py</code>	3-class in-situ labelling; epoch-timestamp filenames; re-entrant	
TFLite inference	128×128 float32 pipeline; 3-class argmax at ≈ 15 FPS	
Flask API	<code>/state</code> returns state/confidence/fps; O(1) latency	
Delta-tracking	Firebase writes only on state transition; sustained safe = 0 writes	
Firebase publish	<code>bridge.py</code> writes root /; UART parser writes <code>vehicles/car_01</code>	
HuggingFace	Model & dataset published; URL cited in README	
Training notebook	<code>lane_training.ipynb</code> with metrics, curves, confusion matrix	
systemd services	Both services start on boot; self-heal within 5 s of crash	
Lane-assist cmd	PIC receives 'L'/'C'/'R'; motor direction correct	
Mobile app	Live dashboard; servo command round-trip <300 ms	

A Raspberry Pi File Inventory

Table 11: Raspberry Pi project file inventory

File	Type	Role
<code>cam_test.py</code>	Python utility	Hardware validation: camera init, ISP transform, AEC warm-up
<code>collect_data.py</code>	Python utility	In-situ labelled dataset collection (3-class keyboard-driven)
<code>live.py</code>	Python service	TFLite vision core + Flask <code>/state</code> micro-service API
<code>bridge.py</code>	Python service	Delta-tracking Firebase synchronisation agent
<code>lane_training.ipynb</code>	Jupyter notebook	Full ML fine-tuning pipeline: data splits, augmentation, training curves, confusion matrix, per-class metrics
<code>lane_model.tflite</code>	ML asset	TFLite 3-class lane classifier (128×128 float32 input)
<code>model_config.json</code>	Config	Class list and input image resolution
<code>firebase_credentials.json</code>	Credentials	GCP service-account RSA-2048 key (Admin SDK auth)
<code>lane.service</code>	systemd unit	Vision service: boot startup, <code>Restart=always</code>
<code>firebase-bridge.service</code>	systemd unit	Bridge service: depends on <code>lane.service</code> , <code>Restart=always</code>
<code>requirements.txt</code>	Dependencies	Python package versions for reproducibility

B Glossary

argmax	Mathematical operation returning the index of the maximum value in a vector — used to select the highest-probability class from the TFLite output tensor.
AEC / AWB	Automatic Exposure Control / Automatic White Balance — camera ISP algorithms that converge on ambient lighting; require ≥ 2 s warm-up.
daemon thread	A background thread automatically terminated when the main process exits, preventing zombie threads on shutdown.

delta-tracking	State-caching pattern that suppresses redundant operations by comparing the current value against the previously processed value.
DMA	Direct Memory Access — hardware mechanism transferring sensor data to RAM without CPU involvement, enabling the zero-copy camera pipeline.
float32 normalisation	Conversion of integer pixel values in $[0, 255]$ to floating-point values in $[0.0, 1.0]$ for neural network input compatibility.
GIE	Global Interrupt Enable — bit 7 of INTCON on PIC16F877A; clearing it atomically disables all interrupts, used as a critical-section guard.
GIL	Python Global Interpreter Lock — mutex serialising Python bytecode; provides implicit thread safety for simple dictionary assignments in <code>live.py</code> .
ISP	Image Signal Processor — hardware block applying transforms (flip, crop, exposure) to raw sensor data before pixels reach application memory.
memory-mapped array	Buffer whose backing storage is a DMA region, enabling zero-copy data access from the camera sensor to the numpy inference pipeline.
RBIF	PORTB Change Interrupt Flag — fires when any of RB4–RB7 changes logic level; used for door-sensor edge detection on RB4.
service account	Non-human GCP identity with specific IAM permissions, used by <code>bridge.py</code> to authenticate to Firebase.
TFLite	TensorFlow Lite — reduced-footprint runtime for deploying quantised neural network models on embedded Linux devices such as the Raspberry Pi.
two-pass Timer1	Measurement pattern in <code>ULTRASONIC.c</code> : Pass 1 detects sensor presence (echo-HIGH timeout); Pass 2 measures echo-HIGH duration for round-trip distance.

References

- [1] DARPA, “Urban Challenge Technical Evaluation Criteria,” Defense Advanced Research Projects Agency, 2007. <https://www.darpa.mil/program/urban-challenge>
- [2] C. Caraffi et al., “A system for real-time detection and tracking of vehicles from a single car-mounted camera,” *15th IEEE Intelligent Transportation Systems Conf.*, 2012.
- [3] D. Neven et al., “Towards End-to-End Lane Detection: an Instance Segmentation Approach,” *IEEE Intelligent Vehicles Symposium*, 2018. <https://arxiv.org/abs/1802.05591>
- [4] Z. Qin, H. Wang, and X. Li, “Ultra Fast Structure-aware Deep Lane Detection,” *ECCV*, 2020. <https://arxiv.org/abs/2004.11757>
- [5] Microchip Technology, “PIC16F877A Data Sheet,” DS40001452C, 2003. <https://ww1.microchip.com/downloads/en/DeviceDoc/39582b.pdf>
- [6] Google LLC, “Firebase Realtime Database,” <https://firebase.google.com/docs/database>, 2024.

- [7] M. A. Sadek, “ADAS Lane Detection Model and Dataset,” HuggingFace, 2026. <https://huggingface.co/MohammedAliSadek/adas-lane-detection>